

Project Report

Name: Abhijeet Kumar **Roll Number:** 24f2007359 **Email:** 24f2007359@ds.study.iitm.ac.in

Brief Description: I am a Standalone Degree Student pursuing BS in Data Science.

Demo Video:

https://drive.google.com/file/d/1lpxv3wDccF5ooeRD0YWa_L9hgrayK3j0/view?usp=drive_link

Placement Portal Application V2 — Campus Recruitment Management System

Project Description

The Placement Portal Application V2 (PPA-V2) is a full-stack, role-based web application that digitises the entire campus placement process for an institute. It replaces the usual mix of spreadsheets and manual email threads with a single system in which three kinds of users — the institute Admin, recruiting Companies, and Students — each work from a dedicated dashboard. The backend is a Flask RESTful API and the frontend is a Vue.js 3 single-page application, together forming a complete ecosystem for coordinating placement drives from job posting through to final placement.

The system models the real recruitment pipeline. Companies register and, once approved by the Admin, post placement drives; the Admin approves each drive before students can see it; students browse eligible drives, apply, and track their status through a full audit trail (Applied → Shortlisted → Interview → Offer → Placed / Rejected). Companies move candidates along this pipeline, schedule interviews, and record placements. On top of this, the platform runs asynchronous background jobs for interview reminders, monthly placement reports, and on-demand CSV exports, and it uses Redis caching to keep the most frequently read endpoints fast. Every status change is recorded so that admins, companies, and students each have a complete, role-appropriate view of the history.

Key Features

- JWT-based authentication with role-based access control (Admin / Company / Student)
- Three role-specific dashboards, each gated by server-side RBAC decorators
- Company registration with an Admin approval workflow before dashboard access
- Job posting workflow requiring Admin approval before drives become visible to students
- Student eligibility checks on apply (minimum CGPA, allowed branch, graduation year)
- Duplicate-application prevention enforced by both an application-level check and a database unique constraint
- Full application status tracking with an append-only audit trail (`ApplicationStatusHistory`)
- Placement records created automatically when a candidate is offered or placed

- Interview scheduling, feedback, and offer-letter download
- Admin management: approve / reject / blacklist / remove companies, students, and job postings
- Search of companies (by name / industry) and students (by name / institute ID / contact)
- Asynchronous background jobs using Celery + Redis (interview reminders, reports, exports)
- Scheduled daily interview reminders and monthly per-company placement reports (Celery Beat)
- Monthly placement reports generated as HTML **and PDF**, emailed to each company
- User-triggered CSV export of application and placement history, delivered asynchronously with a completion email
- Redis response caching with namespace-version invalidation (measured 12.5× speedup on job listings)
- Admin cache-control panel (view versions/keys/TTL, flush cache)
- Graceful degradation — the app serves correct data straight from the database if Redis is unavailable
- Email notifications via Gmail SMTP, with a safe console-logging fallback when credentials are unset
- Responsive Vue.js frontend using Bootstrap 5
- Cross-Origin Resource Sharing (CORS) configured for the frontend origin
- X-Cache: HIT | MISS response header for cache observability

Technologies Used

Backend Technologies:

- Flask 3.1.0 — Python web framework for the RESTful API
- Flask-SQLAlchemy 3.1.1 — Flask integration layer for SQLAlchemy
- SQLAlchemy 2.0.36 — ORM for database models and querying
- Flask-Cors 5.0.0 — Cross-Origin Resource Sharing support
- PyJWT 2.10.1 — JSON Web Token creation and verification (custom auth layer)
- Werkzeug 3.1.3 — WSGI utilities and password hashing (script via `generate_password_hash`)
- Celery 5.6.3 — Distributed task queue for background job processing
- Redis 8.0.1 (Python client) — In-memory store used as the Celery broker/result backend and the response cache
- xhtml2pdf 0.2.17 — HTML-to-PDF rendering for monthly placement reports (pure-Python, no system libraries)

- python-dotenv 1.2.2 — Loads mail credentials from `backend/.env`
- SQLite — Development database (schema is portable to PostgreSQL/MySQL)

Frontend Technologies:

- Vue.js 3.5.13 — Progressive JavaScript framework (Composition API, `<script setup>`)
- Vue Router 4.5.0 — SPA routing with navigation guards
- Bootstrap 5.3.3 — Responsive CSS framework
- Axios 1.7.9 — Promise-based HTTP client with request interceptors
- Vite 6.0.7 — Build tool and dev server (`@vitejs/plugin-vue 5.2.1`)
- HTML5 / CSS3 / JavaScript ES6+ — Modern web standards

Development & Deployment Tools:

- Git — Version control
- Python virtual environment — Backend dependency isolation
- NPM — Frontend package management
- Redis server — Cache and Celery broker (run under WSL Ubuntu)
- Environment variables via `.env` — Secure mail configuration (`gitignored`)

DB Schema Design

The database uses a unified `User` table for authentication, with separate profile tables for companies and students, and transactional tables for jobs, applications, placements, and status history. All timestamps are stored in UTC.

User Table (`users`):

- `id` (Integer, Primary Key) — Unique identifier
- `email` (String(120), Unique, Not Null, Indexed) — Login identifier for all roles
- `password_hash` (String(256), Not Null) — Securely hashed password (Werkzeug)
- `role` (Enum: admin / company / student, Not Null) — Determines dashboard and permissions
- `is_active` (Boolean, Default=True, Not Null) — Account kill-switch, re-checked on every request
- `created_at` (DateTime, Default=UTC now, Not Null)
- `company / student` (Relationships) — 1:1 links to the matching profile (None for an admin)

Design Rationale: A single authentication table for all three roles gives one login route, one token shape, and one uniqueness rule on email. Admins have no profile row; companies and students each

have a matching 1:1 profile. Because JWTs are stateless and cannot be revoked, `is_active` is re-read from the database on every authenticated request, providing an immediate kill-switch for blacklisted users who still hold a valid token.

Company Table (`companies`):

- `id` (Integer, Primary Key)
- `user_id` (Integer, FK → `users.id`, Unique, Not Null) — enforces the 1:1 link
- `name` (String(150), Not Null, Indexed), `industry`, `location`, `website`, `hr_contact`, `description`
- `approval_status` (Enum: pending / approved / rejected, Default=pending, Not Null)
- `is_blacklisted` (Boolean, Default=False, Not Null)
- `created_at` (DateTime, Default=UTC now)
- `job_positions` (Relationship, cascade delete-orphan), `placements` (Relationship)

Design Rationale: A company is gated: it cannot access its dashboard or have visible drives until the Admin approves it. `hr_contact` is used to email the monthly report when it looks like an address (otherwise the login email is used). Cascade delete on `job_positions` means removing a company cleanly removes its jobs and, transitively, their applications.

Student Table (`students`):

- `id` (Integer, Primary Key)
- `user_id` (Integer, FK → `users.id`, Unique, Not Null)
- `full_name` (String(150), Not Null, Indexed)
- `institute_id` (String(50), Unique, Indexed) — optional but unique when present
- `contact`, `branch`, `cgpa` (Float), `graduation_year` (Integer), `skills`, `education`, `experience`, `resume_path`
- `is_blacklisted` (Boolean, Default=False), `created_at` (DateTime)
- `applications` (Relationship, cascade delete-orphan), `placements` (Relationship)

Design Rationale: `branch`, `cgpa`, and `graduation_year` form the "eligibility trio" tested when a student applies to a job with eligibility rules. All three are nullable, so a partially completed profile simply cannot apply to gated drives, with a clear message telling the student what to fill in.

JobPosition Table (`job_positions`):

- `id` (Integer, Primary Key)
- `company_id` (Integer, FK → `companies.id`, Not Null, Indexed)
- `title` (String(200), Not Null, Indexed), `description`, `salary_min`, `salary_max`
- `skills_required`, `experience_required`, `benefits`

- `eligibility_branch` (comma-separated list; the word "all" admits everyone), `eligibility_cgpa`, `eligibility_year`
- `application_deadline` (DateTime)
- `status` (Enum: pending / approved / active / closed, Default=pending, Not Null)
- `created_at` (DateTime), `applications` (Relationship, cascade delete-orphan)

Design Rationale: A posting starts `pending` and is invisible to students until the Admin approves it — and the Admin cannot approve a job whose company is not itself approved. `approved` and `active` both count as "students may apply"; `closed` is reused to represent an admin-rejected drive.

Application Table (`applications`):

- `id` (Integer, Primary Key)
- `student_id` (FK → `students.id`, Not Null, Indexed), `job_id` (FK → `job_positions.id`, Not Null, Indexed)
- `status` (Enum: applied / shortlisted / interview / offer / rejected / placed, Default=applied, Not Null)
- `applied_at` (DateTime, Not Null), `feedback` (Text), `interview_date` (DateTime)
- **Unique constraint on (`student_id`, `job_id`)**
- `placement` (1:1 Relationship), `status_history` (1:n Relationship, cascade delete-orphan, ordered)

Design Rationale: The core transactional entity linking a student to a job. The unique constraint is the real guarantee against duplicate applications; the route also checks first for a friendly error, but under a race the constraint decides. `interview_date` is the column the daily reminder job scans. `status` holds the current state; the full path taken is recorded in `status_history`.

Placement Table (`placements`):

- `id` (Integer, Primary Key)
- `student_id` (FK, Not Null, Indexed), `company_id` (FK, Not Null, Indexed), `application_id` (FK, Unique)
- `position` (String(200), Not Null), `salary` (Integer), `joining_date` (Date), `offer_letter_path`
- `created_at` (DateTime) — used to bucket placements by month for reports

Design Rationale: Created or updated when a company marks a candidate `offer` or `placed`. The unique `application_id` enforces a 1:1 relationship with the application, so re-marking a candidate updates the existing row rather than creating duplicates.

ApplicationStatusHistory Table (`application_status_history`):

- `id` (Integer, Primary Key), `application_id` (FK, Not Null, Indexed)
- `from_status` (Enum, nullable for the first entry), `to_status` (Enum, Not Null)
- `changed_by_role` (String), `changed_by_user_id` (FK → `users.id`), `note` (Text), `created_at` (DateTime)

Design Rationale: An append-only audit trail — one row per status change — that satisfies the requirement for complete application history. `from_status` is null only on the first "applied" entry. Roles are stored as strings so history survives user deletion.

Database Relationships and Integrity

- **User → Company / Student (One-to-One):** each company/student user has exactly one profile
- **Company → JobPosition (One-to-Many, cascade delete):** a company owns many drives
- **Student → Application (One-to-Many, cascade delete):** a student files many applications
- **JobPosition → Application (One-to-Many, cascade delete):** a drive receives many applications
- **Application → Placement (One-to-One, optional):** an application yields at most one placement
- **Application → ApplicationStatusHistory (One-to-Many, cascade delete):** each application keeps its full timeline

The schema follows third-normal-form principles, uses foreign-key constraints for referential integrity, and relies on SQLAlchemy's parameterised queries to prevent SQL injection. All timestamps use UTC.

API Architecture

Authentication Endpoints (`/api/auth`):

- `POST /api/auth/register/student` — Student self-registration (usable immediately)
- `POST /api/auth/register/company` — Company registration (created as pending)
- `POST /api/auth/login` — JWT issuance for all roles
- `GET /api/auth/me` — Current user profile (JWT required)

Admin Endpoints (`/api/admin`):

- `GET /api/admin/dashboard` — System stats and approval-queue counters
- `GET /api/admin/companies` — Search companies (`q`, `industry`, `status`)
- `PUT /api/admin/companies/<id>/approve | reject | blacklist`,
`DELETE /api/admin/companies/<id>`

- GET /api/admin/students — Search students (name / ID / contact)
- PUT /api/admin/students/<id>/blacklist | deactivate | activate, DELETE /api/admin/students/<id>
- GET /api/admin/students/<id> — Profile + applications + placements
- GET /api/admin/jobs, PUT /api/admin/jobs/<id>/approve | reject, DELETE /api/admin/jobs/<id>
- GET /api/admin/applications, GET /api/admin/applications/<id> — List and detail (with history)
- GET /api/admin/placements — All placement records
- GET /api/admin/cache/stats, POST /api/admin/cache/flush — Cache introspection and flush
- POST /api/admin/reports/monthly, POST /api/admin/reports/company/<id>, POST /api/admin/reminders/interviews — Manual job triggers
- GET /api/admin/reports, GET /api/admin/reports/download/<filename> — Report listing and download

Company Endpoints (/api/company):

- GET /api/company/dashboard — Company stats (gated by approval)
- GET /api/company/jobs, POST /api/company/jobs, PUT /api/company/jobs/<id> — Manage own postings
- GET /api/company/jobs/<id>/applications — Applicants for one job
- GET /api/company/applications, GET /api/company/applications/<id> — Received applications and detail
- PUT /api/company/applications/<id>/status — Move a candidate along the pipeline (logs history, upserts placement)
- GET /api/company/students/<id> — Applicant profile (only for students who applied to the company)
- GET /api/company/placements — Own placement records

Student Endpoints (/api/student):

- GET /api/student/dashboard — Personal stats
- GET /api/student/profile, PUT /api/student/profile, POST /api/student/profile/resume — Profile and resume upload
- GET /api/student/jobs — Browse approved drives (q, company)

- `POST /api/student/jobs/<id>/apply` — Apply (eligibility + duplicate checks)
- `GET /api/student/applications`, `GET /api/student/applications/<id>` — Track applications (detail includes history)
- `GET /api/student/placements`, `GET /api/student/applications/<id>/offer-letter`

Export Endpoints (`/api/exports`):

- `POST /api/exports/applications`, `POST /api/exports/placements` — Start async CSV export (returns `task_id`)
- `GET /api/exports/status/<task_id>` — Poll job state
- `GET /api/exports/download/<filename>` — Download the generated CSV (ownership-checked)

System Endpoint: `GET /api/health` — Liveness probe.

Background Jobs and Automation

Scheduled Jobs (Celery Beat):

1. **Interview Reminders (`send_interview_reminders`)** — Runs daily at 09:00 IST. Finds every application in `interview` status whose interview falls within the next 24 hours and emails the student the position, company, and time. The 24-hour window matched to a once-a-day cadence ensures each interview triggers exactly one reminder.
2. **Monthly Placement Reports (`generate_monthly_placement_reports`)** — Runs on the 1st of each month at 06:00 IST, reporting on the previous calendar month. For every approved company it computes application/placement statistics and a conversion rate, renders an HTML report, converts it to PDF, and emails it to the company.

Asynchronous Tasks (Celery Workers):

- `export_applications_csv(user_id)` — Role-aware export of a student's or company's applications, emailed as a CSV attachment
- `export_placements_csv(user_id)` — Placement history export (admins receive all placements)
- `generate_company_report(company_id)` — On-demand single-company report

Each user-triggered job returns a `task_id` immediately; the frontend polls the status endpoint until completion, and the user also receives a completion email — satisfying the requirement that an alert is sent once a batch job finishes.

Caching Strategy:

- Job listings cached with a 60-second TTL; admin company/student search cached with a 120-second TTL
- Redis-based cache (database 2, kept separate from the Celery broker and result backend)

- **Namespace-version invalidation:** each key embeds a version number; invalidating a whole group is a single atomic INCR, avoiding an O(n) KEYS scan
- **Expiry policy** (TTL as a backstop) plus **refresh policy** (explicit invalidation on every write) together
- Per-user cache keys for endpoints whose response varies by caller (e.g. a student's `already_applied` flag), preventing cross-user data exposure
- Measured performance: **18.31 ms** → **1.47 ms (12.5× faster)** on a 60-job listing
- Graceful degradation: if Redis is unavailable, every read falls back to the database and writes still succeed

Frontend Architecture

Component Structure (`src/views/`):

- `LoginView.vue` — Single login form for all three roles
- `RegisterStudentView.vue`, `RegisterCompanyView.vue` — Role-specific registration
- `AdminDashboard.vue` — Companies, Students, Job Postings, Applications, Placements, and Reports & Jobs tabs (including the cache panel)
- `CompanyDashboard.vue` — My Jobs, Post Job, Applications (pipeline actions), and Placements tabs, with timeline/profile/placement modals
- `StudentDashboard.vue` — Profile, Browse Jobs, My Applications (with status-timeline modal), and Placements tabs

Service Layer (`src/services/`):

- `api.js` — Axios instance with a request interceptor that attaches the JWT to every call
- `auth.js` — Reactive auth store (login, register, logout, current user)
- `admin.js`, `company.js`, `student.js` — Typed API wrappers per role
- `exports.js` — Async export helper: triggers a job, polls to completion, and downloads the file

Routing (`src/router/index.js`):

- Public routes: Login, Register (Student / Company)
- Protected routes: role-specific dashboards
- A global navigation guard enforces authentication and role, with a matching server-side RBAC layer as the true boundary

Development Workflow

Environment Setup:

1. Backend virtual environment with `pip install -r requirements.txt`

2. Mail credentials configured in `backend/.env` (loaded by `python-dotenv`; gitignored)
3. `python seed_admin.py` to create tables and seed the predefined Admin
4. Redis server started for caching and as the Celery broker
5. Frontend dependencies installed with `npm install`

Service Management (development):

- Flask API on port 5000 (`python app.py`)
- Vue.js dev server on port 5173 with hot reload (`npm run dev`, proxying `/api` to Flask)
- Celery worker for background task execution (`celery -A celery_app.celery worker`)
- Celery Beat for scheduled jobs (`celery -A celery_app.celery beat`)
- Redis on port 6379

Production Considerations:

- A WSGI server (e.g. Gunicorn) for the Flask app and `npm run build` for the optimised frontend bundle
- Environment-based configuration with secure secret keys and mail credentials
- Redis persistence and Celery worker scaling based on load
- A reverse proxy (e.g. Nginx) serving static assets and routing API calls, with HTTPS

Security Features

- Password hashing using Werkzeug (`generate_password_hash / check_password_hash`)
- Stateless JWT authentication (HS256), with `is_active` re-checked on every request as a revocation mechanism
- Role-based access control enforced server-side via `@role_required` decorators (the frontend guard is UX only)
- CORS restricted to the frontend origin
- SQL-injection prevention through SQLAlchemy parameterised queries
- Path-traversal protection on file downloads (`secure_filename` plus a resolved-path containment check)
- Ownership checks on exports (a user can only download files stamped with their own id) and on applicant profile access (a company can only view students who applied to its jobs)
- Per-user cache keys to prevent one user being served another user's cached response
- Input validation on both the frontend and backend
- Mail credentials kept out of the repository in a gitignored `.env` file

Testing Strategy

- Automated end-to-end test suites for each major milestone driving the real HTTP layer via Flask's test client:
 - Milestone 6 (application history and placements): 22 assertions
 - Milestone 7 (Celery jobs, PDF/CSV generation): 27 assertions, including valid PDF output
 - Milestone 8 (Redis caching): 36 assertions, including cache hit/miss, per-user isolation, invalidation on every write path, TTL, error responses not being cached, and correct behaviour when Redis is down
- A dedicated benchmark measuring cached vs. uncached latency (the 12.5× figure above)
- A database-fingerprint isolation test ensuring separate databases never share cache entries
- Live integration testing with a running Redis instance and a real Celery worker
- Manual UI testing of the full flow: registration, approvals, job posting, application, the status pipeline, exports, and reports

Milestone Completion Summary

Milestone	Feature	Status
M0	GitHub repository setup	Done
M1	Database models & schema	Done
M2	Authentication & role-based access	Done
M3	Admin dashboard & management	Done
M4	Company dashboard & job/application management	Done
M5	Student dashboard & job application system	Done
M6	Application history & status tracking	Done
M7	Celery + Redis background jobs (reminders, reports, exports)	Done
M8	API optimization & Redis caching	Done

All eight core milestones are complete.

Conclusion

The Placement Portal Application V2 demonstrates a complete, role-based implementation of a campus recruitment system built with a Flask backend and a Vue.js 3 frontend. It faithfully models the real placement pipeline with layered approval gates, enforces access control on the server side, and records a full audit trail of every application's progress. Beyond the synchronous features, it

adds asynchronous automation with Celery and Redis — interview reminders, monthly PDF reports, and async CSV exports — and a Redis caching layer that measurably improves response times while degrading gracefully when the cache is unavailable. The modular blueprint-based backend and service-oriented frontend keep the codebase maintainable and ready for the optional enhancements (analytics charts, an ATS resume screener, and PWA features).

Declaration of AI Usage

AI assistance (Anthropic's Claude, via the Claude Code CLI) was used during the development of this project. It was used for generating and refactoring backend and frontend code, writing explanatory code comments and documentation, designing the Redis caching and background-job architecture, and writing automated test suites. All AI-generated code was reviewed, tested, and integrated by me, and I understand the complete functioning of the submitted codebase.

Estimated AI/LLM Used %: ~20% %